TỰ ĐỘNG TẠO BÀI THUYẾT TRÌNH SỬ DỤNG CÁC MÔ HÌNH NGÔN NGỮ LỚN VÀ MÔ HÌNH NGÔN NGỮ ĐA PHƯƠNG THỨC

Nguyễn Trung Kiên[1], Phạm Văn Minh[1], Nguyễn Minh Quang[1], Vũ Thành Tuyên[1], Đặng Hoàng Long [1]

[1] Học viện Công nghệ Bưu chính Viễn thông

kienbk16082004@gmail.com, mata.pv01@gmail.com, nguyenquang71103@gmail.com, tuyenvt455@gmail.com, longdh@ptit.edu.vn

TÓM TẮT—Bài thuyết trình là một công cụ quan trọng để truyền tải thông tin trực quan và hiệu quả nhưng việc tạo ra các bài thuyết trình chất lượng cao thường đòi hỏi kỹ năng trình bày nâng cao và tốn nhiều thời gian, công sức. Các phương pháp tự động hóa hiện tại chủ yếu tập trung vào chuyển đổi văn bản thành slide, thường bỏ qua yếu tố trực quan, dẫn đến các slide thiếu sáng tạo và không đủ sức thu hút khán giả. Để khắc phục hạn chế này, chúng tôi đề xuất SlideGen, một hệ thống tự động tạo bài thuyết trình từ các tài liệu như bài báo, báo cáo, etc. Nghiên cứu của chúng tôi đã tích hợp mô hình ngôn ngữ lớn (LLM) và mô hình ngôn ngữ đa phương thức (VLM) với kỹ thuật Function Calling, cho phép tạo ra các slide độc đáo, sáng tạo, và phù hợp với nhiều mục đích khác nhau. Kết quả cho thấy phương pháp của chúng tôi đã giúp giảm đáng kể thời gian và công sức của người dùng, đồng thời tạo ra các bài thuyết trình với thiết kế trực quan hấp dẫn và nội dung phù hợp, mở ra tiềm năng ứng dụng trong giáo dục, kinh doanh, và nghiên cứu khoa học, etc.

Từ khóa— Bài thuyết trình(Presentation), mô hình ngôn ngữ lớn(Large language models - LLMs), mô hình ngôn ngữ đa phương thức(Vision language models - VLMs), gọi hàm(Function Calling)

I. GIỚI THIỆU

Hiện nay, các bài thuyết trình là một phương tiện truyền tải thông tin được sử dụng rộng rãi, được đánh giá cao nhờ hiệu quả trực quan trong việc thu hút và giao tiếp với khán giả. Tuy nhiên, việc tạo ra các bài thuyết trình chất lượng cao đòi hỏi một câu chuyện hấp dẫn, bố cục được thiết kế tốt và nội dung phong phú, lôi cuốn [1]. Do đó, việc tạo ra các bài thuyết trình hoàn chỉnh đòi hỏi kỹ năng trình bày nâng cao và rất nhiều công sức. Trước sự phức tạp vốn có của quá trình này, ngày càng có nhiều sự quan tâm đến việc tự động hóa quá trình tạo bài thuyết trình [2, 3, 4] bằng cách khai thác khả năng tổng quát hóa của các mô hình ngôn ngữ lớn và mô hình ngôn ngữ đa phương thức. Các công cụ truyền thống như Microsoft PowerPoint hay Canva chỉ hỗ trợ định dạng nội dung, khiến việc chuẩn bị nội dung vẫn phụ thuộc nặng nề vào nỗ lực thủ công.

Các phương pháp hiện tại chủ yếu dựa trên mô hình chuyển đổi văn bản thành slide, tận dụng các quy tắc hoặc mẫu có sẵn để chuyển đổi kết quả từ mô hình ngôn ngữ lớn thành các slide. Theo các nghiên cứu trước đây [4, 5], các cách tiếp cận này thường coi việc tạo bài thuyết trình như một quá trình tóm tắt nội dung, chú trọng vào phần văn bản mà bỏ qua yếu tố trực quan vốn là cốt lõi của bài thuyết trình. Hệ quả là những bài thuyết trình này thường quá tải văn bản, thiếu sự đa dạng và không thể thu hút khán giả một cách hiệu quả [6].

Chúng tôi đã nghiên cứu và đề xuất một phương pháp tự động tạo ra các bài thuyết trình cho các tài liệu, bài báo, báo cáo sử dụng các mô hình ngôn ngữ lớn và mô hình ngôn ngữ đa phương thức, kết hợp với các kỹ thuật hỗ trợ. Trước tiên, chúng tôi tích hợp kỹ thuật Function Calling gọi tới thư viện các hàm HTML để tạo ra các slide độc đáo, không phụ thuộc vào các template cố định. Sau đó, phương pháp này tận dụng cơ chế đánh giá của mô hình ngôn ngữ đa phương thức để tối ưu hóa chất lượng slide thông qua việc điều chỉnh prompt và tái tạo nội dung, đảm bảo các slide đạt được tính sáng tạo và thiết kế hấp dẫn nhất. Phương pháp này cho phép tạo ra các bài thuyết trình sáng tạo hơn, với thiết kế thu hút và nội dung phù hợp, đồng thời giảm đáng kể thời gian và công sức của người dùng.

Trong phần II, chúng tôi sẽ khảo sát các nghiên cứu liên quan đến tự động hóa tạo bài thuyết trình. Phần III trình bày chi tiết phương pháp đề xuất. Phần IV mô tả các kết quả thực nghiệm. Cuối cùng, phần V đưa ra kết luận, những hạn chế hiện tại và định hướng phát triển trong tương lai.

II. CÁC NGHIÊN CỨU LIÊN QUAN

Một slide thuyết trình bao gồm hai thành phần chính: nội dung văn bản truyền tải thông tin cốt lõi và thiết kế trực quan bao quanh, bao gồm hình ảnh, biểu đồ, và bố cục, nhằm tăng tính thẩm mỹ và dễ hiểu. Trong bài báo này, khi nhắc đến "bài thuyết trình tự động", chúng tôi quy ước nó bao gồm cả nội dung văn bản và thiết kế trực quan. Mặc dù cấu trúc slide có vẻ đơn giản, việc tạo ra một bài thuyết trình chất lượng cao đòi hỏi nhiều thời gian và công sức, từ việc tổ chức nội dung logic đến thiết kế giao diện hấp dẫn. Do đó, tự động hóa tạo bài thuyết trình đã thu hút sự quan tâm lớn nhằm giảm thiểu công sức, đồng thời đáp ứng nhu cầu tạo ra các slide đa dạng, chất lượng cao, phù hợp với nhiều đối tượng và mục đích, từ học thuật đến kinh doanh. Tuy nhiên, thách thức lớn nhất trong quá trình này là yêu cầu các hệ thống trí tuệ nhân tạo phải hiểu ngữ nghĩa và ngữ cảnh của tài liệu đầu vào, đồng thời kết hợp hài hòa giữa nội dung văn bản và yếu tố trực quan để tạo ra các slide vừa chính xác về thông tin vừa lôi cuốn về hình thức.

Để giải quyết những khó khăn trong tự động hóa tạo bài thuyết trình, nhiều nghiên cứu đã được tiến hành. Ban đầu, các phương pháp học máy truyền thống được áp dụng, như ROUGE của Lin và các cộng sự [7], một công cụ đánh giá tóm tắt văn bản, nhưng không phù hợp để đánh giá yếu tố trực quan của slide. Nhóm nghiên cứu của Duarte [8, 9] đưa ra các nguyên tắc thiết kế thủ công, nhấn mạnh tính hấp dẫn thị giác và câu chuyện lôi cuốn, nhưng khó lập trình hóa để tự động hóa quy mô lớn. Sau đó, các phương pháp sử dụng mô hình ngôn ngữ lớn đã được phát triển. Nhóm nghiên cứu của Guo [10] giới thiệu bộ đánh giá PPTC để đo lường khả năng xử lý văn bản của LLMs trong các tác vụ PowerPoint, trong khi Cachola và các cộng sự[12] đề xuất phương pháp tạo khuôn mẫu tài liệu để tổ chức nội dung slide. Năm 2024, nhóm nghiên cứu của Liu[11] sử dụng GPT-4 để cải thiện chất lượng nội dung slide, và Chen[13] áp dụng MLLM-as-a-judge để đánh giá slide đa phương thức. Nhìn chung, các nghiên cứu trước đây chủ yếu tập trung vào xử lý văn bản hoặc sử dụng khuôn mẫu cố định, dẫn đến hạn chế về tính sáng tạo và khả năng tạo ra các slide trực quan hấp dẫn. Bên cạnh đó, việc thiếu một bộ dữ liệu tiêu chuẩn và phương pháp đánh giá toàn diện khiến việc so sánh hiệu quả giữa các phương pháp trở nên khó khăn. Hơn nữa, các hệ thống hiện tại thường gặp thách thức về hiệu năng khi xử lý các tài liệu đầu vào phức tạp, dẫn đến kết quả không đồng đều.

Để tự xây dựng một hệ thống sinh bài thuyết trình tự động hoàn chỉnh chắc chắn sẽ vô cùng tốn kém, cùng với đó là những yêu cầu về sự đầu tư cho tính đa dạng của các mẫu slide. Thời gian gần đây, các mô hình ngôn ngữ lớn và mô hình ngôn ngữ đa phương thức đã và đang thể hiện sức mạnh vượt trội, vì vậy chúng tôi đã lên ý tưởng tận dụng những sức mạnh đó để thực hiện công việc này nhằm tiết kiệm thời gian và công sức trong việc ra các bài thuyết trình.

III. PHƯƠNG PHÁP ĐƯỢC ĐỀ XUẤT

A. Function Calling

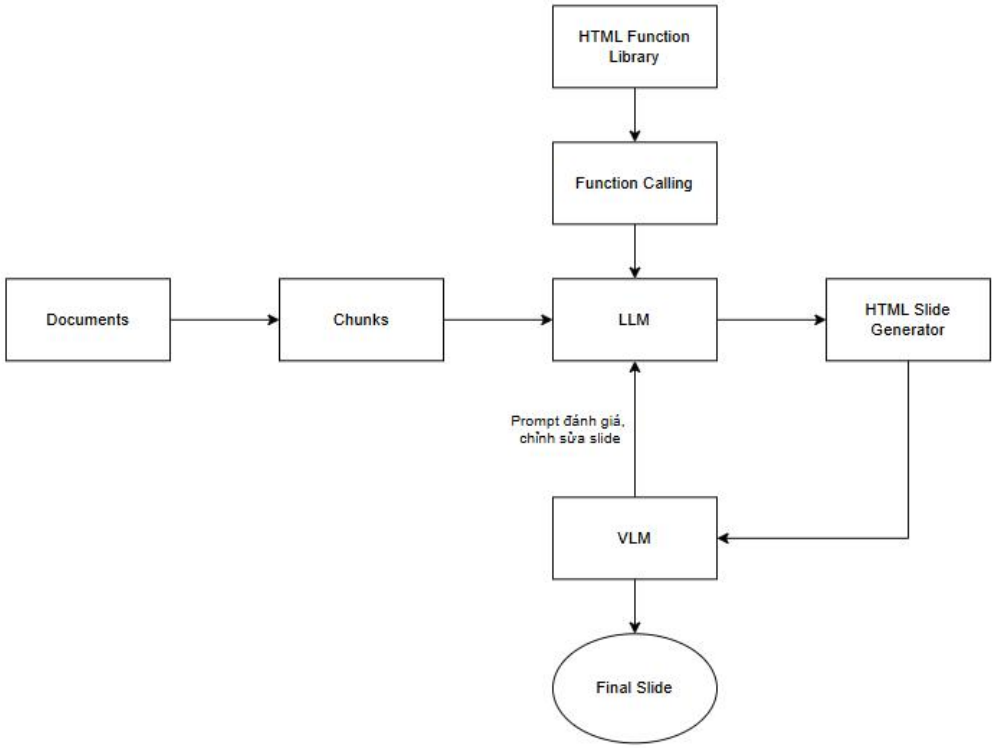
Trong bối cảnh các mô hình ngôn ngữ lớn, Function Calling được định nghĩa là khả năng của mô hình trong việc hiểu các yêu cầu của người dùng và thực thi các thủ tục tính toán được chỉ định để cung cấp các câu trả lời chính xác, phù hợp với ngữ cảnh. Công nghệ này nâng cao đáng kể độ chính xác của các hệ thống AI khi xử lý các truy vấn phức tạp, đồng thời cho phép kết nối trực tiếp với nhiều nguồn tài nguyên bên ngoài, bao gồm cơ sở dữ liệu, luồng thông tin thời gian thực, giao diện ứng dụng bên ngoài, các mô hình ngôn ngữ tiên tiến khác, và các dịch vụ bổ sung. Chẳng hạn, các hệ thống này có thể lấy thông tin thời tiết hiện tại thông qua tích hợp với dịch vụ khí tượng hoặc thực thi các đoạn mã lập trình nhờ cơ chế diễn giải mã linh hoạt. Ngoài ra, khả năng này hỗ trợ các quá trình phân tích tức thời và ra quyết định hoạt động, đóng vai trò quan trọng trong nhiều lĩnh vực chuyên môn như nghiên cứu tài chính, hệ thống thông tin y tế, và các giải pháp hỗ trợ cá nhân hóa, etc. Nhờ vậy, Function Calling đã thay đổi căn bản kiến trúc của các nền tảng trí tuệ tự động và cách chúng tương tác với người dùng.

Trong phương pháp của chúng tôi, kỹ thuật Function Calling được tận dụng để tối ưu hóa LLM nhằm tạo ra các slide bài thuyết trình chất lượng cao, đáp ứng các yêu cầu cụ thể của người dùng. Phương pháp này cho phép LLM phân tích dữ liệu đầu vào (tài liệu DOCX) và gọi các hàm được định nghĩa sẵn trong một thư viện chuyên dụng để tạo ra các slide với nội dung và bố cục tùy chỉnh. Khác với các phương pháp truyền thống phụ thuộc vào các template cố định, Function Calling kết hợp xử lý nội dung với thực thi hành động cụ thể, tạo ra các kết quả linh hoạt và sáng tạo. Theo tìm hiểu của chúng tôi, hiện nay chưa có nghiên cứu nào áp dụng Function Calling vào việc tự động hóa tạo bài thuyết trình.

B. Sử dụng các mô hình ngôn ngữ lớn và mô hình ngôn ngữ đa phương thức trong tự động tạo bài thuyết trình

Để tạo ra các trang slide cho tài liệu đầu vào, đầu tiên cần xử lý tài liệu và chia nó thành các phần được gọi là các chunk. Các chunk này cùng với các tool sẽ được đưa vào LLM để quyết định nội dung của các trang slide. LLM sẽ đọc hiểu và thực hiện sửa đổi như là tóm gọn, bổ sung, nhấn mạnh các nội dung quan trọng và thực hiện việc lựa chọn các hàm phù hợp với nội dung và chủ đề của slide hiện tại để thực hiện Function Calling.

Tools được định nghĩa từ trước là phần mô tả chi tiết của các hàm HTML/CSS trong HTML Library đã được xây dựng từ trước, giúp LLM hiểu và nắm bắt được một cách chi tiết về các hàm (chủ đề, bố cục, các biến, etc.) giúp việc lựa chọn hàm và căn chỉnh slide lúc LLM Function Calling được thực hiện một cách chính xác và hợp lý. Ngoài ra, LLM còn được cung cấp thêm lời gọi hàm trước đó (nếu có) để tham khảo và giúp slide được tạo ra một cách đồng bộ và nhất quán, tránh hỗn loạn về mặt thẩm mỹ. Slide được LLM tạo ra sẽ được đưa vào VLM dưới dạng ảnh cùng với ảnh của slide trước đó để VLM thực hiện đánh giá theo một số tiêu chí nhất định, nếu được VLM chấp nhận thì mới cho ra kết quả cuối cùng, còn ngược lại thì VLM sẽ góp ý cho LLM và yêu cầu LLM tạo lại slide mới.



Hình 1. Mô hình tự động tạo bài thuyết trình với LLM, Function Calling và VLM

Hình 1 mô tả quy trình SlideGen tiếp nhận và xử lý tài liệu đầu vào theo thứ tự, kết quả thu được là tập các slide được cho là hoàn thiện nhất. Thành phần của SlideGen bao gồm mô hình ngôn ngữ lớn LLM, mô hình ngôn ngữ đa phương phức VLM, thư viện các hàm HTML. Trong đó chúng tôi áp dụng Function Calling giúp LLM có thể tạo ra các slide một cách sáng tạo nhưng theo 1 bố cục rõ ràng và hợp lý. Mỗi slide sẽ được VLM xem xét một cách tỉ mỉ cùng với slide trước nó để có thể đánh giá và nâng cao chất lượng thẩm mỹ của slide. Quy trình tạo ra một slide đầy đủ gồm có ba bước như sau:

1. Tiền xử lý:

Quy trình bắt đầu bằng bước tiền xử lý, khi người dùng tải lên tài liệu gốc dưới định dạng phổ biến như PDF hoặc DOCX. Tài liệu này sẽ được lưu trữ và phân loại tại lớp nhập liệu nhằm tạo nền tảng cho các bước xử lý tiếp theo. SlideGen sau đó áp dụng kỹ thuật chia nhỏ (chunking) để phân tách tài liệu thành các đoạn nội dung ngắn, phù hợp cho việc phân tích. Các đoạn nội dung này được trích xuất, gán nhãn và sắp xếp theo thứ tự logic, hình thành một danh sách nội dung rõ ràng cho từng slide. Việc tổ chức nội dung theo cách này không chỉ đảm bảo tính mạch lạc mà còn giúp mô hình ngôn ngữ dễ dàng tiếp cận và xử lý thông tin hiệu quả hơn trong các bước tiếp theo.

2. Các bước xử lý của LLM để tạo slide:

Sau khi hoàn tất khâu tiền xử lý, danh sách nội dung sẽ được đưa vào mô hình ngôn ngữ lớn (LLM) để thực hiện quá trình tạo slide. LLM đảm nhiệm các tác vụ đọc hiểu, phân tích, tóm tắt và xác định các điểm chính cần trình bày trên từng slide. Dựa trên đặc điểm nội dung, LLM sẽ lựa chọn mẫu HTML/CSS phù hợp từ HTML Library nhằm đảm bảo sự tương thích giữa thông tin và hình thức trình bày. Thông qua Function Calling, LLM không chỉ tạo nội dung văn bản cho từng slide mà còn trực tiếp điền các thông tin đã xử lý vào mẫu HTML/CSS tương ứng. Bên cạnh đó, LLM có khả năng điều chỉnh các yếu tố thẩm mỹ như màu sắc, phông chữ, bố cục, hình nền và hiệu ứng thị giác, từ đó tạo ra các slide không chỉ rõ ràng về nội dung mà còn có phong cách trình bày sáng tạo và đa dạng. Việc tinh chỉnh này giúp mỗi slide mang tính cá nhân hóa cao, phù hợp với mục đích truyền đạt và thẩm mỹ tổng thể của bài thuyết trình. Kết quả là các slide được định dạng hoàn chỉnh, đảm bảo cả về mặt nội dung lẫn hình thức, và sẵn sàng cho bước đánh giá tiếp theo.

3. Các bước xử lý của VLM để đánh giá và cải thiện chất lượng slide:

Sau khi LLM hoàn tất việc tạo nội dung slide, VLM sẽ tiếp nhận để đánh giá và cải thiện chất lượng từng slide. Toàn bộ nội dung HTML/CSS được tổng hợp lại bởi HTML Slide Generator để tạo thành giao diện trình chiếu hoàn chỉnh, trực quan và thân thiện với người dùng. VLM tiến hành phân tích từng slide cùng với slide liền kề để đưa ra đánh giá toàn diện dựa trên các tiêu chí như mức độ rõ ràng, tính thẩm mỹ, sự cân đối thị giác và mức độ phù hợp

với mục tiêu thuyết trình. Nếu phát hiện slide chưa đạt yêu cầu, VLM sẽ tự động điều chỉnh prompt đầu vào và kích hoạt lại quá trình tạo slide, từ đó hình thành một vòng lặp cải tiến liên tục. Cơ chế phản hồi này cho phép hệ thống không ngừng hoàn thiện từng slide, đảm bảo đầu ra cuối cùng là một bài thuyết trình có chất lượng cao cả về mặt nội dung lẫn hình thức.

IV. KẾT QUẢ THỰC NGHIỆM

A. Phương pháp đánh giá

Do hiện nay chưa có một thước đo tiêu chuẩn nào để đánh giá chất lượng của các slide được tạo tự động, chúng tôi tiến hành thu thập đánh giá theo cách thủ công từ các giảng viên, sinh viên trong trường cùng với những người làm việc trong môi trường công sở. Các tiêu chí chúng tôi đặt ra bao gồm đánh giá thiết kế, đánh giá chức năng và đánh giá hiệu năng.

Đánh giá thiết kế: Thiết kế của hệ thống SlideGen được đánh giá dựa trên ba yếu tố chính: tính thẩm mỹ, độ rõ ràng và dễ đọc, cùng với tính nhất quán. Về tính thẩm mỹ, SlideGen chú trọng vào sự hài hòa trong phối màu, bố cục cân đối và việc sử dụng hình ảnh phù hợp để thu hút người xem. Ví dụ, một slide với màu nền tối và chữ sáng sẽ tạo độ tương phản tốt, giúp nội dung nổi bật mà không gây khó chịu cho mắt. Độ rõ ràng và dễ đọc được đảm bảo thông qua việc chọn font chữ dễ nhìn, kích thước chữ phù hợp để đọc từ xa, khoảng cách dòng hợp lý và độ tương phản cao giữa chữ và nền. Cuối cùng, tính nhất quán đảm bảo rằng các slide trong bài thuyết trình duy trì phong cách thiết kế đồng bộ về font, màu sắc và bố cục, mang lại vẻ chuyên nghiệp và giúp người xem tập trung vào nội dung thay vì bị phân tâm bởi sự thay đổi hình thức.

Đánh giá chức năng: Chức năng của SlideGen được đánh giá qua hai khía cạnh: độ phù hợp của hàm được gọi và tính hiệu quả của kỹ thuật Function Calling. Độ phù hợp của hàm được gọi thể hiện ở việc các hàm HTML/CSS được hệ thống sử dụng có đáp ứng đúng mục đích của từng slide hay không. Chẳng hạn, một slide giới thiệu cần hàm tạo tiêu đề lớn và hình ảnh nền nổi bật, trong khi slide dữ liệu cần hàm sinh biểu đồ hoặc bảng. Điều này đảm bảo nội dung được trình bày đúng với ý định ban đầu. Tính hiệu quả của Function Calling được đánh giá qua khả năng của mô hình ngôn ngữ lớn trong việc tự động chọn và thực thi hàm chính xác từ thư viện HTML Functions Library mà không cần chỉnh sửa thủ công.

Đánh giá hiệu năng: Hiệu năng của SlideGen được đo lường dựa trên thời gian xử lý và tài nguyên sử dụng. Thời gian xử lý là khoảng thời gian hệ thống cần để tạo ra một bài thuyết trình hoàn chỉnh từ tài liệu đầu vào. Một thời gian xử lý ngắn sẽ là lợi thế lớn, đặc biệt trong các tình huống cần hoàn thành gấp, như chuẩn bị cho cuộc họp đột xuất. Về tài nguyên sử dụng, SlideGen được đánh giá qua lượng CPU, GPU và bộ nhớ tiêu tốn trong quá trình hoạt động. Một hệ thống tối ưu sẽ tiêu tốn tài nguyên ở mức hợp lý, cho phép hoạt động trên nhiều thiết bị mà không đòi hỏi cấu hình phần cứng cao, từ đó tăng tính linh hoạt trong triển khai.

B. Tiến hành thử nghiệm

Hệ thống SlideGen đã được triển khai và đánh giá trên nền tảng phần cứng sử dụng GPU NVIDIA A5000 24GB, nhằm đảm bảo khả năng xử lý hiệu quả cho các mô hình lớn. Để hỗ trợ quá trình tạo slide, hệ thống sử dụng một thư viện gồm gần 100 mẫu slide HTML/CSS được thiết kế chuyên nghiệp, cung cấp đa dạng các bố cục và phong cách trình bày nhằm đáp ứng tính linh hoạt và phù hợp với nhiều loại nội dung khác nhau. Trong quá trình thực nghiệm, mô hình ngôn ngữ lớn được triển khai theo hai hướng là sử dụng là Qwen2.5-7B-Instruct (áp dụng kỹ thuật tăng tốc flash_attention_2 để tối ưu hóa hiệu suất) và gọi API Gemini Flash 2.5 để so sánh với nhau.

Các hàm HTML trong HTML Library được thiết kế để làm sao vừa tạo ra một bố cục mẫu rõ ràng, rành mạch, vừa không hạn chế khả năng sáng tạo của LLM/VLM, mỗi một hàm sẽ có một tool tương ứng để giúp LLM/VLM có được cái nhìn tổng quan về hàm đó. Tool là phần mô tả hàm HTML/CSS giúp LLM hiểu được chi tiết hàm đó ví dụ như tên hàm, mô tả hàm (chủ đề, bố cục, phù hợp với nội dung có dạng như thế nào, etc.), tên các tham số, mô tả các tham số, kiểu dữ liệu của các tham số, các tham số bắt buộc, etc. Hình 2 và hình 3 dưới đây là ví dụ về một hàm HTML/CSS đơn giản trong HTML Library và định nghĩa tool tương ứng của nó:

```
def generate_intro_slide(
    main_title="Main Title",
    subtitle1="Subtitle1",
    subtitle2="Subtitle2",
    accent_bg_color="#6ea88a",
```

```
    .main-title {{
        font-size: {main_title_font_size};
        color: {title_color};
        margin: 0 0 15px 0;
        font-weight: bold;
```

```
    title_color="#333333",
    subtitle_color="#555555",
    font_family="'Segoe UI', Arial, sans-serif",
    main_title_font_size="36px",
    subtitle_font_size="16px",
):

<html_code = f"""<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-
width, initial-scale=1.0">
    <title>Marketing Slide</title>
    <style>
        body {{
            margin: 0;
            padding: 0;
            font-family: {font_family};
            background-color: #f5f5f5;
            display: flex;
            justify-content: center;
            align-items: center;
            min-height: 100vh;
        }}

        .slide-container {{
            width: 1000px;
            height: 560px;
                position: relative;
                Background-color:
                {main_bg_color};
            overflow: hidden;
                box-shadow: 0 5px 15px rgba(0, 0, 0, 0.1);
        }}

        .right-section {{
            position: absolute;
            top: 0;
            right: 0;
            width: 40%;
            height: 100%;
              Background-color: {accent_bg_color};
        }}
            line-height: 1.2;
        }}

        .subtitle {{
            font-size: {subtitle_font_size};
            color: {subtitle_color};
            margin: 5px 0;
        }}
    </style>
</head>
<body>
    <div class="slide-container">
        <!-- Left and right sections -->
        <div class="left-section"></div>
        <div class="right-section"></div>

        <!-- Small green bar at top left -->
        <div class="green-bar-top"></div>

        <!-- Green bar across the middle -->
        <div class="green-bar-middle"></div>

        <!-- Text box -->
        <div class="text-box">
            <h1 class="main-title">{main_title}</h1>
            <p class="subtitle">{subtitle1}</p>
            <p class="subtitle">{subtitle2}</p>
            <div class="divider"></div>
        </div>
    </div>
</body>
</html>"""

    return html_code
```

Hình 2. Ví dụ về một hàm HTML trong HTML Funtion Library

```
{
  "name": "generate_intro_slide",
  "description": "Generates an HTML introduction slide
with a  main title section on the left and an image on the
right.",
  "parameters": {
    "type": "object",
    "properties": {
      "main_title": {
        "type": "string",
        "description": "Main title text (e.g., 'Main Title')."
      },
      "subtitle1": {
        "type": "string",
      "title_color": {
          "type": "string",
          "description": "CSS color for the main title (e.g.,
       '#333333')."
          },
       "subtitle_color": {
          "type": "string",
          "description": "CSS color for the subtitles (e.g.,
       '#555555')."
          },
      "font_family": {
          "type": "string",
          "description": "CSS font family for the slide (e.g.,
'Segoe UI, Arial, sans-serif')."
```

```
      "description": "First subtitle text (e.g., 'Subtitle1')."
    },
    "subtitle2": {
      "type": "string",
      "description": "Second subtitle text (e.g., 'Subtitle2')."
    },
    "accent_bg_color": {
      "type": "string",
      "description": "CSS background color for the right section (e.g., '#6ea88a')."
    },
    },
    "main_title_font_size": {
      "type": "string",
      "description": "CSS font size for the main title (e.g., '36px')."
    },
    "subtitle_font_size": {
      "type": "string",
      "description": "CSS font size for the subtitles (e.g., '16px')."
    },
    "required": ["main_title", "subtitle1", "subtitle2"]
  }
}
```

Hình 3. Ví dụ về tool tương ứng cho hàm HTML ở trên

Dưới đây là ví dụ về một vài slide được tạo ra dựa trên phương pháp của chúng tôi với đầu vào là một tài liệu có chủ đề về cây xanh.



Hình 4. Ví dụ về một số slide được tạo ra theo phương pháp của chúng tôi

Kết quả thực nghiệm cho thấy phương pháp của chúng tôi có khả năng tạo ra các slide đáp ứng tốt các tiêu chí về tính trực quan, thẩm mỹ và phù hợp nội dung, đồng thời giảm đáng kể thời gian và công sức của người dùng so với các phương pháp thủ công. Ngoài ra, để so sánh hiệu quả giữa việc chạy local Qwen 2.5 7B và gọi api Gemini 2.5 Flash làm LLM và VLM cho SlideGen, chúng tôi có bảng so sánh dưới đây.

Bảng 1. Kết quả đánh giá chất lượng các slide được tạo tự động

Tiêu chí	Qwen 2.5 7B-Instruct (flash_attention_2)	Call API Gemini 2.5 Flash	Ghi chú
Độ phù hợp của nội dung slide	Tỷ lệ slide có nội dung phù hợp: 85%	Tỷ lệ slide có nội dung phù hợp: 95%	Đánh giá dựa trên đánh giá của người dùng hoặc chuyên gia.

Tính thẩm mỹ	Điểm trung bình đánh giá của người dùng: 3.9/5	Điểm trung bình đánh giá của người dùng: 4.5/5	Đánh giá dựa trên khảo sát người dùng hoặc đánh giá của chuyên gia thiết kế.
Độ rõ ràng và dễ đọc	Tỷ lệ slide có độ rõ ràng và dễ đọc tốt: 78%	Tỷ lệ slide có độ rõ ràng và dễ đọc tốt: 87%	Đánh giá dựa trên các tiêu chí về font chữ, kích thước chữ, độ tương phản.
Độ phù hợp của hàm được gọi	Tỷ lệ hàm được gọi phù hợp: 89%	Tỷ lệ hàm được gọi phù hợp: 94%	Đánh giá dựa trên kiểm tra thủ công hoặc tự động.
Thời gian xử lý	Thời gian trung bình để tạo một bài thuyết trình (10 slide): 600 giây	Thời gian trung bình để tạo một bài thuyết trình (10 slide): 180 giây	So sánh thời gian cần thiết để xử lý cùng một tài liệu.
Tài nguyên sử dụng	GPU memory: 24GB CPU memory: 16GB	GPU memory: 0GB CPU memory: 4GB	So sánh lượng tài nguyên được sử dụng.

Dựa trên bảng so sánh, có thể thấy việc gọi api Gemini 2.5 Flash có hiệu suất tốt hơn chạy local Qwen 2.5 7B ở hầu hết các hạng mục, đặc biệt là về thời gian xử lý và độ phù hợp của nội dung slide. Tuy nhiên, chi phí bỏ ra cho việc call api thì sẽ tốn hơn, và việc gọi api Gemini 2.5 Flash sẽ không tối ưu do không thể thực hiện finetune cho LLM hay VLM. Vì vậy, trong tương lai nhóm nghiên cứu sẽ sử dụng api Gemini 2.5 Flash để tạo dataset và finetune lại Qwen 2.5 7B, finetune và sử dụng LM để chia nội dung thay vì sử dụng chunking, dự định mở rộng HTML Functions Library để tạo thêm nhiều slide sinh động, hấp dẫn hơn, đồng thời hỗ trợ xử lý nhiều định dạng đầu vào với các cấu trúc khác nhau, nhằm nâng cao tính linh hoạt và khả năng ứng dụng. Một hạn chế hiện tại là chưa tối ưu hóa hoàn toàn cho các tài liệu chứa nội dung đa phương thức phức tạp, và đây sẽ là trọng tâm nghiên cứu tiếp theo.

V. KẾT LUẬN

Bài báo đã trình bày về một phương pháp tự động tạo bài thuyết trình từ tài liệu đầu vào bằng cách tích hợp các mô hình ngôn ngữ lớn và mô hình ngôn ngữ đa phương thức với kỹ thuật Function Calling. Phương pháp này, kết hợp với HTML Functions Library, cho phép tạo ra các slide linh hoạt, sáng tạo, và phù hợp với nhiều mục đích khác nhau. Trong tương lai, chúng tôi dự định mở rộng HTML Functions Library để tạo thêm nhiều slide sinh động, hấp dẫn hơn, đồng thời hỗ trợ xử lý nhiều định dạng đầu vào với các cấu trúc khác nhau, nhằm nâng cao tính linh hoạt và khả năng ứng dụng. Một hạn chế hiện tại là chưa tối ưu hóa hoàn toàn cho các tài liệu chứa nội dung đa phương thức phức tạp, và đây sẽ là trọng tâm nghiên cứu tiếp theo.

VI. TÀI LIỆU THAM KHẢO

[1] Tsu-Jui Fu, William Yang Wang, Daniel McDuff, and Yale Song. 2022. Doc2ppt: Automatic presentation slides generation from scientific documents. Proceedings of the AAAI Conference on Artificial Intelligence, 36(1):634–642.

[2] Jiaxin Ge, Zora Zhiruo Wang, Xuhui Zhou, Yi-Hao Peng, Sanjay Subramanian, Qinyue Tan, Maarten Sap, Alane Suhr, Daniel Fried, Graham Neubig, et al. 2025. Autopresent: Designing structured visuals from scratch. arXiv preprint arXiv:2501.00912.

[3] Himanshu Maheshwari, Sambaran Bandyopadhyay, Aparna Garimella, and Anandhavelu Natarajan. 2024. Presentations are not always linear! gnn meets llm for document-to-presentation transformation with attribution. arXiv preprint arXiv:2405.13095.

[4] Ishani Mondal, S Shwetha, Anandhavelu Natarajan, Aparna Garimella, Sambaran Bandyopadhyay, and Jordan Boyd-Graber. 2024. Presentations by the humans and for the humans: Harnessing llms for generating persona-aware slides from documents. In Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers), pages 2664–2684.

[5] Athar Sefid, Prasenjit Mitra, and Lee Giles. 2021. Slidegen: an abstractive section-based slide generator for scholarly documents. In Proceedings of the 21st ACM Symposium on Document Engineering, pages 1–4.

[6] Andrea Barrick, Dana Davis, and Dana Winkler. 2018. Image versus text in powerpoint lectures: Who does it benefit? Journal of Baccalaureate Social Work, 23(1):91–109.

[7] Chin-Yew Lin. 2004. ROUGE: A package for automatic evaluation of summaries. In Text Summarization Branches Out, pages 74–81, Barcelona, Spain. Association for Computational Linguistics

[8] Nancy Duarte. 2008. Slide: ology: The art and science of creating great presentations, volume 1. O'Reilly Media Sebastapol.

[9] Nancy Duarte. 2010. Resonate: Present visual stories that transform audiences. John Wiley & Sons.

[10] Yiduo Guo, Zekai Zhang, Yaobo Liang, Dongyan Zhao, and Duan Nan. 2023. Pptc benchmark: Evaluating large language models for powerpoint task completion. arXiv preprint arXiv:2311.01767.

[11] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. 2023. G-eval: NLG evaluation using gpt-4 with better human alignment. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pages 2511–2522, Singapore. Association for Computational Linguistics.

[12] Isabel Alyssa Cachola, Silviu Cucerzan, Allen Herring, Vuksan Mijovic, Erik Oveson, and Sujay Kumar Jauhar. 2024. Knowledge-centric templatic views of documents. In Findings of the Association for Computational Linguistics: EMNLP 2024, pages 15460–15476, Miami, Florida, USA. Association for Computational Linguistics.

[13] Dongping Chen, Ruoxi Chen, Shilin Zhang, Yinuo Liu, Yaochen Wang, Huichi Zhou, Qihui Zhang, Pan Zhou, Yao Wan, and Lichao Sun. 2024a. Mllm as-a-judge: Assessing multimodal llm-as-a-judge with vision-language benchmark. arXiv preprint arXiv:2402.04788.

AUTOMATIC PRESENTATION GENERATION USING LLMs AND VLMs

Nguyen Trung Kien, Pham Van Minh, Nguyen Minh Quang, Vu Thanh Tuyen, Dang Hoang Long

ABSTRACT*— Presentations are an important tool for conveying information in a visual and effective manner. However, creating high-quality presentations often requires advanced presentation skills and consumes a significant amount of time and effort. Current automation methods mainly focus on converting text into slides, often neglecting visual elements, resulting in slides that lack creativity and fail to engage the audience. To address this limitation, we propose SlideGen, a system that automatically generates presentations from documents such as articles, reports, etc. Our research integrates Large Language Models (LLMs) and Vision-Language Models (VLMs) with Function Calling techniques, enabling the creation of unique, creative slides tailored for various purposes. The results show that our method significantly reduces users' time and effort while producing visually appealing presentations with relevant content. This opens up great potential for applications in education, business, scientific research, and more.*